Hub de Estudos

Apostilas, exercícios e simulados

JSON



HTTP



Segurança



API

Desenvolvimento de APIs para a Web

REST, Next.js, Prisma e segurança

Desenvolvimento de APIs para a Web

REST, Next.js, Prisma e segurança

Hub de Estudos

Apostilas, exercícios e simulados

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia.

Desenvolvido por TecPro Tecnologias.

Sumário

Capítulo 1 – APIs e Web Services **4**
1.1 O que é uma API 4
1.2 Cliente, servidor e o protocolo HTTP 4
1.3 A anatomia de uma URL 6
1.4 Serviços web com SOAP 6
1.5 APIs REST 7
1.6 GraphQL 7
1.7 Outros estilos de comunicação 8
1.8 Comparando SOAP, REST e GraphQL 8
1.9 Consultando uma API pública 9
1.10 O projeto da apostila: a API do TaskFlow 10
Erros Comuns 11

Capítulo 1 – APIs e Web Services

Quando você abre o aplicativo do banco e vê o seu saldo, quando um site de compras calcula o frete a partir do CEP ou quando um sistema emite uma nota fiscal eletrônica, há sempre dois programas conversando: um que pede uma informação e outro que a entrega. Essa conversa acontece por meio de uma **API**. Neste capítulo você vai entender o que é uma API, como funciona o protocolo HTTP, que transporta essas conversas na web, e quais são os principais estilos de serviço web: SOAP, REST e GraphQL. No final, vai consultar uma API pública de verdade e conhecer o projeto que será construído ao longo da apostila.

1.1 O que é uma API

API é a sigla de *Application Programming Interface*, ou interface de programação de aplicações. É um conjunto de regras que define como um programa pode pedir algo a outro: quais pedidos existem, que dados cada pedido precisa receber e o que vem de volta.

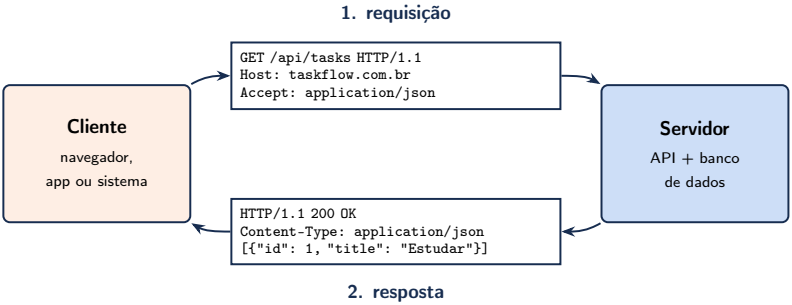
Uma comparação comum é a do restaurante. O cliente não entra na cozinha: ele consulta o cardápio e faz o pedido ao garçom, que leva o pedido à cozinha e traz o prato. O cardápio é o **contrato** da API (o que pode ser pedido), o garçom é a API, e a cozinha é o sistema que faz o trabalho. O cliente não precisa saber como a cozinha funciona, e a cozinha pode mudar de fogão sem que o cliente perceba, desde que o cardápio continue o mesmo.

Esse é o grande valor de uma API: ela **separa quem usa de quem implementa**. Um mesmo servidor pode atender, ao mesmo tempo, o site, o aplicativo de celular e o sistema de um parceiro, todos usando o mesmo contrato.

O termo API é amplo. As funções de uma biblioteca, como o `readFile` do Node.js, também formam uma API. Nesta apostila, porém, o foco são as **APIs web**: aquelas acessadas pela rede, usando o protocolo HTTP. Quando essas APIs são oferecidas por um sistema para ser usadas por outros sistemas, também são chamadas de **serviços web** (*web services*).

1.2 Cliente, servidor e o protocolo HTTP

Toda conversa com uma API web segue o modelo **cliente-servidor**: o **cliente** envia uma **requisição** (*request*) e o **servidor** devolve uma **resposta** (*response*). O cliente pode ser um navegador, um aplicativo de celular ou outro servidor. As regras dessa troca são as do **HTTP** (*HyperText Transfer Protocol*), o mesmo protocolo que o navegador usa para carregar páginas.



o servidor não guarda memória entre uma requisição e outra (sem estado)

Uma requisição HTTP tem quatro partes:

- o **método**, que diz o que o cliente quer fazer: `GET` para buscar, `POST` para criar, `PUT` e `PATCH` para alterar, `DELETE` para excluir;
- a **URL**, que diz em que recurso a ação será feita, como `/api/tasks`;
- os **cabeçalhos** (*headers*), pares nome e valor com informações extras, como o formato aceito na resposta;
- o **corpo** (*body*), opcional, com os dados enviados, por exemplo os campos de uma tarefa nova.

A resposta tem três: o **código de status**, um número que resume o resultado (`200` deu certo, `404` não encontrado, `500` erro no servidor), os **cabeçalhos** e o **corpo**, com os dados pedidos. Por baixo de tudo, requisição e resposta são apenas texto. Veja como fica a criação de uma tarefa:

```
POST /api/tasks HTTP/1.1
Host: taskflow.com.br
Content-Type: application/json

{"title": "Estudar HTTP", "priority": "HIGH"}
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/tasks/42

{"id": 42, "title": "Estudar HTTP", "priority": "HIGH"}
```

A linha em branco separa os cabeçalhos do corpo. O cabeçalho `Content-Type` informa o formato do corpo; aqui, `application/json`. O status `201 Created` indica que um recurso novo foi criado, e o cabeçalho `Location` diz onde ele pode ser encontrado. Os métodos e os códigos de status são estudados em detalhe nos Capítulos 2 e 8.

Uma característica essencial do HTTP é ser **sem estado** (*stateless*): cada requisição é independente, e o servidor não se lembra das anteriores. Se o servidor precisa saber quem está pedindo, a própria requisição tem de trazer essa informação, normalmente

um **token** em um cabeçalho. É assim que funciona o login em APIs, assunto do Capítulo 9.

1.3 A anatomia de uma URL

A URL (*Uniform Resource Locator*) identifica onde está o recurso. Cada parte tem uma função:

```
https://api.taskflow.com.br:443/v1/tasks?status=DONE&page=2
```

Parte	Exemplo	Função
protocolo	`https`	como a conversa acontece; o `s` indica conexão criptografada
domínio	`api.taskflow.com.br`	o nome do servidor
porta	`443`	a "porta de entrada"; 443 é o padrão do HTTPS e costuma ser omitida
caminho	`/v1/tasks`	o recurso dentro do servidor
query string	`?status=DONE&page=2`	parâmetros extras, como filtros e paginação, em pares nome=valor separados por `&`

Na *query string*, espaços e acentos não podem aparecer como são: eles são codificados, e "São Paulo" vira `S%C3%A3o%20Paulo`. O `fetch` e as bibliotecas de HTTP cuidam disso com `encodeURIComponent` ou com a classe `URLSearchParams`.

1.4 Serviços web com SOAP

Os primeiros serviços web populares, no início dos anos 2000, usavam o **SOAP** (*Simple Object Access Protocol*), um protocolo padronizado pelo W3C. No SOAP, toda mensagem é um documento **XML** com uma estrutura fixa, o **envelope**, e o serviço é descrito em um arquivo **WSDL** (*Web Services Description Language*), também em XML, que lista as operações, os parâmetros e os tipos de cada uma. A partir do WSDL, ferramentas geram automaticamente o código do cliente.

```
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
    xmlns:t="http://taskflow.com.br/ws">
  <soap:Header/>
  <soap:Body>
    <t:BuscarTarefa>
      <t:id>42</t:id>
    </t:BuscarTarefa>
  </soap:Body>
</soap:Envelope>
```

Repare que a operação, `BuscarTarefa`, aparece dentro da mensagem. No SOAP, todas as chamadas costumam ir para uma mesma URL, quase sempre com o método `POST`, e é o conteúdo do envelope que diz o que fazer. O padrão também define como devolver

erros (o elemento `Fault`) e tem extensões para segurança e transações, como o WS-Security.

O SOAP é verboso e rígido, mas essa rigidez é justamente o que alguns sistemas querem: um contrato formal, validado automaticamente. Por isso ele continua presente em bancos, seguradoras e em sistemas de governo. No Brasil, a emissão da Nota Fiscal Eletrônica (NF-e) é feita por serviços web SOAP das secretarias da Fazenda.

1.5 APIs REST

O **REST** (*Representational State Transfer*) não é um protocolo, e sim um **estilo de arquitetura**, descrito por Roy Fielding em sua tese de doutorado, em 2000. A ideia central é organizar a API em torno de **recursos**, as "coisas" do sistema (tarefas, usuários, pedidos), cada um identificado por uma URL, e usar os **métodos do próprio HTTP** para dizer o que fazer com eles:

```
GET    /api/tasks       lista as tarefas
POST   /api/tasks       cria uma tarefa
GET    /api/tasks/42    busca a tarefa 42
PATCH  /api/tasks/42    altera parte da tarefa 42
DELETE /api/tasks/42    exclui a tarefa 42
```

O servidor devolve uma **representação** do recurso, hoje quase sempre em **JSON**, um formato de texto leve e fácil de ler tanto por pessoas quanto por programas:

```
{
  "id": 42,
  "title": "Estudar HTTP",
  "status": "PENDING",
  "priority": "HIGH",
  "dueDate": "2026-11-20"
}
```

Compare com o SOAP: em vez de uma operação `BuscarTarefa` escrita dentro de um envelope, o REST usa `GET` na URL da tarefa. As regras do estilo, como a ausência de estado e o uso de URLs para identificar recursos, são o tema do Capítulo 2. Por ser simples, usar o HTTP como ele foi pensado e funcionar bem com JavaScript, o REST se tornou o estilo dominante nas APIs web, e é o estilo usado nesta apostila.

1.6 GraphQL

O **GraphQL** foi criado pelo Facebook em 2012 e aberto ao público em 2015. Ele é uma **linguagem de consulta** para APIs: em vez de várias URLs, há um único endereço, normalmente `/graphql`, e o cliente envia uma consulta dizendo exatamente quais campos quer receber.

```
query {
  usuario(id: 7) {
    nome
    tarefas(status: PENDING) {
```

```
      title
      dueDate
    }
  }
}
```

```
{
  "data": {
    "usuario": {
      "nome": "Ana",
      "tarefas": [
        { "title": "Estudar HTTP", "dueDate": "2026-11-20" }
      ]
    }
  }
}
```

A resposta tem o mesmo formato da consulta. Isso resolve dois problemas comuns do REST: receber campos demais, que o cliente não vai usar (*over-fetching*), e precisar de várias requisições para montar uma tela, como uma para o usuário e outra para as tarefas dele (*under-fetching*). Em troca, o servidor fica mais complexo, e recursos simples do HTTP, como o *cache* por URL, deixam de funcionar sozinhos. O GraphQL brilha em aplicações com telas muito variadas, como redes sociais; para a maioria das APIs, o REST continua mais simples.

1.7 Outros estilos de comunicação

Três outras formas de comunicação aparecem com frequência ao lado das APIs:

- **gRPC**: criado pelo Google, usa o HTTP/2 e um formato binário, o *Protocol Buffers*, muito mais compacto que o JSON. É comum na comunicação entre servidores de uma mesma empresa, em que o desempenho importa mais que a facilidade de leitura.
- **WebSocket**: abre uma conexão que fica aberta, e o servidor pode enviar dados ao cliente a qualquer momento, sem esperar uma requisição. É a base de chats, placares ao vivo e notificações em tempo real.
- **Webhook**: é o caminho inverso de uma API. Em vez de o seu sistema perguntar de tempos em tempos "o pagamento foi aprovado?", o serviço de pagamento faz uma requisição `POST` para uma URL do seu sistema assim que o pagamento é aprovado.

1.8 Comparando SOAP, REST e GraphQL

	SOAP	REST	GraphQL
o que é	protocolo	estilo de arquitetura	linguagem de consulta
formato	XML (envelope)	qualquer um; quase sempre JSON	JSON
endereços	uma URL por serviço	uma URL por recurso	uma URL só
operação indicada	no envelope	pelo método HTTP	na consulta
contrato	WSDL, obrigatório	opcional (OpenAPI, Capítulo 10)	esquema, obrigatório
uso típico	bancos, governo, sistemas legados	APIs públicas e a maioria dos sistemas web	telas com dados muito variados

Nenhum estilo é melhor em tudo. A escolha depende de quem vai usar a API, do tipo de dado e das exigências do sistema. Um mesmo produto pode, inclusive, usar mais de um: REST para o aplicativo, SOAP para falar com o sistema fiscal e WebSocket para as notificações.

1.9 Consultando uma API pública

A melhor forma de entender uma API é usar uma. O **ViaCEP** é um serviço gratuito que devolve o endereço de um CEP e não exige cadastro. Como a consulta usa o método `GET`, basta colar a URL no navegador:

```
https://viacep.com.br/ws/01001000/json/
```

Pelo terminal, a ferramenta `curl` faz a mesma requisição e mostra a resposta. Ela já vem instalada no Windows 10 e 11, no macOS e no Linux; no PowerShell, digite `curl.exe`. A opção `-i` mostra também a linha de status e os cabeçalhos:

```
$ curl -i https://viacep.com.br/ws/01001000/json/
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "complemento": "lado ímpar",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP",
  "ddd": "11"
}
```

A resposta real tem mais campos e cabeçalhos; aqui aparecem só os principais. Em um programa, a requisição é feita com o `fetch`, que existe no navegador e, desde a versão 18, também no Node.js:

```
// cep.mjs  -- rode com: node cep.mjs
const cep = "01001000";
const url = `https://viacep.com.br/ws/${cep}/json/`;

const resposta = await fetch(url);
console.log(resposta.status);     // 200

const endereco = await resposta.json();
console.log(`${endereco.logradouro}, ${endereco.localidade}`);
// Praça da Sé, São Paulo
```

O `fetch` devolve uma *Promise* com a resposta; `resposta.status` é o código de status, e `resposta.json()` lê o corpo e o converte em um objeto JavaScript. Troque `json` por `xml` no fim da URL e o mesmo serviço responde em XML, assunto do Capítulo 5.

Experimente também um CEP que não existe, como `99999999`. O ViaCEP responde com status `200` e o corpo `{"erro": "true"}`. É uma escolha de projeto discutível: o Capítulo 8 mostra por que, em uma API REST, o mais adequado seria o status `404`.

1.10 O projeto da apostila: a API do TaskFlow

Ao longo da apostila, você vai construir a API do **TaskFlow**, um gerenciador de tarefas em que cada pessoa cria uma conta e organiza as próprias tarefas. A cada capítulo, a API ganha uma parte nova, até ser publicada na internet com domínio próprio. As ferramentas são as mesmas usadas por empresas no dia a dia:

- **Next.js** com **TypeScript**, para criar as rotas da API (Capítulos 3 e 4);
- **PostgreSQL** na nuvem, com o NeonDB, e o **Prisma** para acessar o banco (Capítulos 3 e 7);
- **Zod**, para validar os dados recebidos (Capítulo 6);
- **JWT**, para a autenticação (Capítulo 9);
- **Vitest**, para os testes automatizados (Capítulo 11);
- **Vercel**, para a publicação (Capítulo 12).

Ao final, a API terá estes *endpoints*, ou seja, as combinações de método e URL que ela atende:

Método	URL	O que faz
`POST`	`/api/auth/register`	cria uma conta
`POST`	`/api/auth/login`	entra e recebe um token
`GET`	`/api/tasks`	lista as tarefas de quem está logado, com filtros
`POST`	`/api/tasks`	cria uma tarefa
`GET`	`/api/tasks/:id`	busca uma tarefa
`PATCH`	`/api/tasks/:id`	altera uma tarefa
`DELETE`	`/api/tasks/:id`	exclui uma tarefa

O `:id` indica uma parte variável da URL: `/api/tasks/42` é a tarefa 42. Para acompanhar, você vai precisar do Node.js (versão 20 ou mais nova), do VS Code e de uma conta gratuita no GitHub, que também serve para entrar no NeonDB e na Vercel.

Erros Comuns

- **Achar que API é sinônimo de REST.** REST é um estilo; SOAP, GraphQL e gRPC também são formas de construir APIs.
- **Achar que o servidor "lembra" do cliente.** O HTTP é sem estado: cada requisição precisa trazer tudo o que o servidor precisa saber, inclusive quem está pedindo.
- **Confundir a URL com o *endpoint*.** O mesmo caminho, `/api/tasks`, é um *endpoint* com `GET` (listar) e outro com `POST` (criar).
- **Esquecer o `await` no `fetch`.** Sem ele, a variável guarda uma *Promise*, e não a resposta. O mesmo vale para `resposta.json()`.
- **Confiar apenas no status.** Algumas APIs, como o ViaCEP com um CEP inexistente, respondem `200` e indicam o erro no corpo. Leia a documentação de cada API.
- **Colocar acentos e espaços direto na *query string*.** Use `encodeURIComponent` ou `URLSearchParams` para montar os parâmetros.